1 · Ponto de partida: por que o Covertype e o que precisamos treinar

Trocamos de dataset para ter um problema com espaço para aplicar a disciplina; o Desafio 3 pergunta como treinar bem a rede que esse problema comporta

Por que mudamos de dataset

	Dataset inicial	Forest Covertype (atual)
Objetivo	Recomendar a cultura a plantar, a partir das condições de solo e clima	**Prever a cobertura vegetal** de uma célula de terreno, a partir de relevo, solo e distâncias
Linhas	2.200	**581.012**
Classes	22 culturas	**7 coberturas vegetais**
Acerto no teste	98,6% (chute: 4,5%)	91,3% na baseline (chute: 48,8%)
Situação	Saturado: já acertava quase tudo	Há margem para melhorar

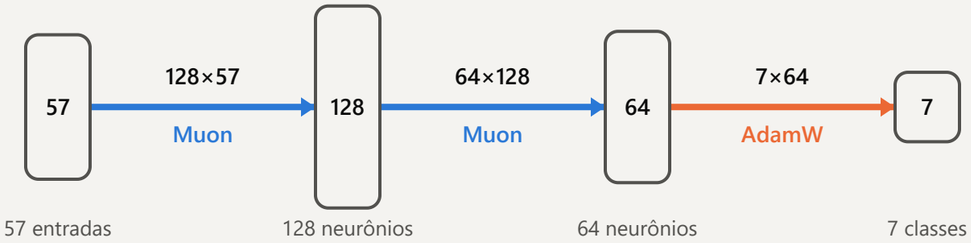
O problema do dataset inicial: era pequeno e fácil demais. Com 98,6% de acerto, qualquer mudança no treino seria invisível, e uma rede maior não cabia na regra de ~20 exemplos de treino por parâmetro.

O que o Covertype dá: volume para uma rede de tamanho razoável dentro dessa regra, classes desbalanceadas como no mundo real e margem para evoluir. Queríamos dar mais **espaço para aplicar os conceitos da disciplina** (inicialização, ativação, otimizadores).

O problema e a rede que vamos treinar

Cada exemplo é uma célula de terreno de 30×30 m de uma floresta dos EUA (Colorado), descrita por **elevação, distâncias até a água e a focos de incêndio, sombreamento, tipo de solo** e outras variáveis. **Objetivo da rede:** dizer qual das **7 coberturas vegetais** domina ali.

cada seta = uma matriz de pesos (e o otimizador que a atualiza)



- **Cada camada oculta** multiplica pelos pesos, soma o viés e aplica a **tanh**. A camada de saída só multiplica e soma, e entrega um **vetor de 7 números (logits)**, um por cobertura: a **posição do maior número** é a cobertura prevista.
- **16.135 parâmetros** (pesos e vieses de cada camada): cerca de **25 exemplos de treino por parâmetro** (406.708). Os 57 valores de entrada vêm das 54 variáveis originais, com as contínuas padronizadas.

Como a rede aprende

Treinar é **ajustar esses 16 mil parâmetros para o erro cair**:

1 · Função de perda: Mede o quanto a rede errou. Transforma os 7 números em probabilidades: é **alta** quando a rede dá pouca probabilidade à cobertura certa e **baixa** quando dá muita.

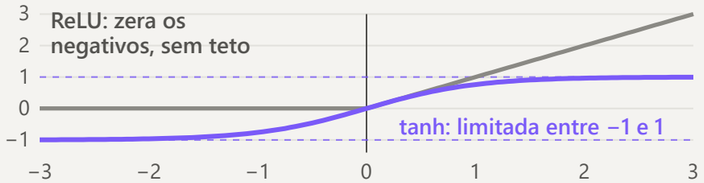
↓

2 · Gradiente: Para cada peso, para onde a perda cresce.

↓

3 · Otimizador: A regra que usa o gradiente para atualizar os pesos a cada passo. O que muda de um para outro é o **tamanho e a direção do passo**: é o que o Desafio 3 compara.

Ativação: trocamos o ReLU pelo tanh (Desafio 2)



- Testamos as ativações com **3 sementes cada**, na validação. A ideia de testar o tanh veio das **apresentações dos colegas** (muita gente o usou).
- O tanh foi a **melhor**: 3 desvios ou mais acima do GELU e do ReLU.
- Ele **limita o sinal entre −1 e 1**, o que ajuda a manter o treino estável.

Baseline: o ponto de comparação

O enunciado pede para manter a **receita atual do grupo**, mas ainda não tínhamos uma, porque tínhamos acabado de trocar de dataset. Então definimos o treino padrão: **Adam, lr 1e-3, agenda constante**. Essa é a nossa **baseline**.

Durante o desenvolvimento, comparamos tudo com ela e buscamos uma **receita melhor** (otimizador + lr + agenda): é o que o Desafio 3 pede.

A pergunta é se existe uma receita melhor e, se existir, **por que** ela é melhor.

2 · O que o desafio pedia e como respondemos

O enunciado define o protocolo em 4 etapas; o que ficou aberto nós decidimos e declaramos · 56 execuções reportadas, nenhuma divergiu

O que o enunciado pede para comparar

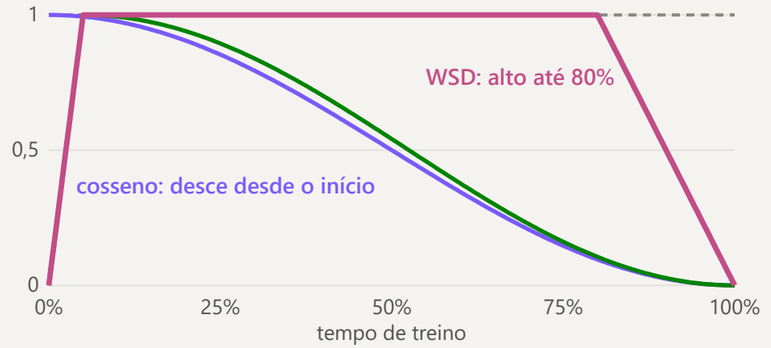
OTIMIZADORES

- **SGD** passo ∝ gradiente
- **SGD Nesterov**
- **Adam** passo normalizado
- **AdamW** Adam + weight decay
- **Muon (obrigatório)** passo ortogonalizado

AGENDAS DE LR

- **constante**
- **cosseno**
- **warmup + cosseno**
- **WSD (nossa escolha)**

Agendas: como o lr muda ao longo do treino (fração do lr máximo)



Cosseno: começa a baixar o lr desde o início, suave, até 0.

WSD (nossa escolha): sobe (warmup), **mantém o lr alto por 80%** do treino e só cai no final.

Escolhemos o WSD porque está nas leituras do enunciado e contrasta com o cosseno. Ele não ganhou, mas mostrou que o que importa é a **queda do lr até perto de 0 no fim**.

Por que 40 épocas em tudo (e 200 só para confirmar)

40 épocas em tudo
o que usamos — **≈ 2,1 h**

200 épocas em tudo
estimativa: 5× o tempo — **≈ 10,5 h**

Tempo de CPU do protocolo inteiro (sem GPU).

O Muon é o mais lento (~9,2× por época). Usamos **40 épocas, igual para todos**; depois **confirmamos com 200 épocas só a vencedora e a baseline**, porque ficava a dúvida se a vantagem do Muon era só aprender mais rápido.

O protocolo do enunciado, em 4 etapas

ETAPA 1
Baseline
A receita do grupo: Adam, lr 1e-3, constante. 3 sementes.

3 execuções
F1 0,8646 ± 0,0028

ETAPA 2
Varredura de lr
5 a 7 lrs por otimizador (warmup + cosseno, 1 semente). O SGD caiu na borda: estendemos.

29 execuções
melhor: Muon, lr 0,03

ETAPA 3
Agendas
As 4 agendas no AdamW e no Muon, cada um no seu melhor lr.

6 execuções
vence warmup + cosseno

ETAPA 4
Confirmação
As melhores, com 3 sementes, mais AdamW e Adam como controle e 200 épocas.

20 execuções
14 de 40 e 6 de 200 épocas

Regras do enunciado e como cumprimos

- ✓ **lr ajustado** por otimizador (Muon sobre o melhor lr do AdamW)
- ✓ **Mesmo orçamento** (passos e lote) em todas as comparações
- ✓ **Validação decide**; teste usado **uma única vez**
- ✓ **Tudo reportado**: nenhuma divergiu; SGD lr 3,0 colapsou (F1 0,61)
- ✓ **Vencedor só se o ganho supera o desvio**: +0,048 contra 0,0009 e 0,0028
- ✓ **Muon incluído** (`torch.optim.Muon`, só nas matrizes ocultas)
- ✓ **Reprodutível**: cada linha do `experimentos.csv` traz o comando exato
- ✓ **Agenda à escolha**: WSD, que contrasta com o cosseno

Outras decisões nossas (registro a cada 25 passos, só o lr ajustado, sementes extras de AdamW e Adam): estão nas notas (tecla N).

3 · O que encontramos: Muon + queda do lr dá +0,048 de F1 sobre a baseline

Do melhor lr de cada otimizador, passando pelas agendas, até a confirmação com 3 sementes

Receita vencedora

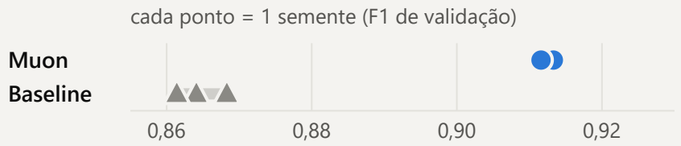
- **Muon**, lr 0,03, nas 2 matrizes ocultas (momento 0,95, Nesterov)
- **AdamW**, lr 0,01, nos vieses e na camada de saída
- agenda **warmup (5%) + cosseno até 0**
- 40 épocas = 31.800 passos, lote 512, igual para todos

Ganho (F1 macro, validação)

40 ÉPOCAS · 3 SEMENTES

0,9121 ± 0,0009

contra **0,8646 ± 0,0028** da baseline (Adam 1e-3, constante)



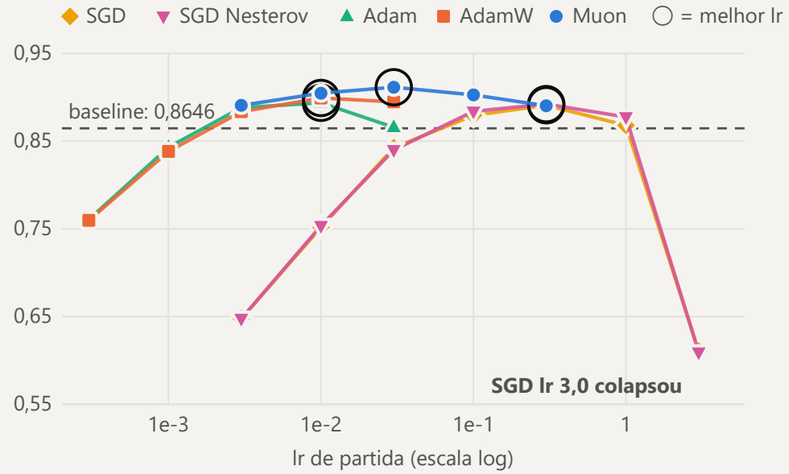
+0,0475 = 16× o desvio entre sementes

Teste (usado uma única vez): F1 **0,9104** contra 0,8689 · acurácia 94,4% contra 91,3%

É a receita adotada no modelo final do projeto (`modelo.py`).

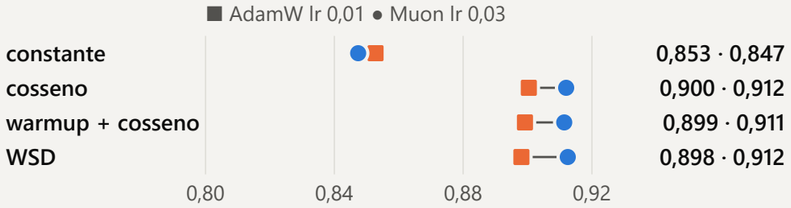
As duas nuvens de pontos não se tocam: a diferença não é sorte de semente.

Etapa 2 · o melhor lr de cada otimizador



- **Melhor lr:** Muon 0,03 (0,911) · AdamW 0,01 (0,899) · Adam 0,01 (0,894) · SGD Nesterov 0,3 (0,892) · SGD 0,3 (0,891). Todos **no interior** da grade.
- O SGD com lr 3,0 **colapsou** (F1 0,61): é a execução instável (página 4). 1 semente por ponto: diferenças menores que ~0,003 não separam efeito de ruído.

Etapa 3 · as agendas (F1, 1 semente)



Com lr **constante** o Muon perde do AdamW (**0,847** contra **0,853**): a vantagem do Muon exige a queda do lr. O WSD ficou abaixo do warmup + cosseno nos dois.

Etapa 4 · confirmação (3 sementes)



- **Vencedora:** Muon lr 0,03 + warmup/cosseno, **0,9121 ± 0,0009** contra 0,8646 ± 0,0028 da baseline: **16× o desvio**, então há vencedor.
- Cosseno puro empata (0,9111 ± 0,0006): a diferença é menor que o desvio, então o **warmup não fez diferença** aqui. O WSD fica 0,0035 abaixo.
- AdamW (0,8994) contra Adam (0,8957) no mesmo lr e agenda: o weight decay soma +0,0037.

4 · Por que funciona: o que os sinais mostram e onde o ganho para de valer

O diagnóstico lido no treino, a decomposição do ganho e as limitações, inclusive a execução que falhou

De onde vem o ganho



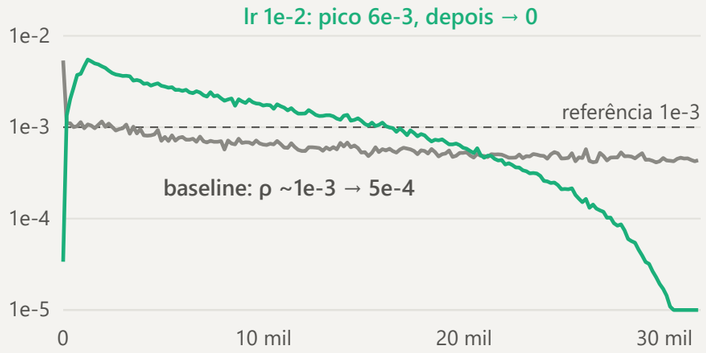
1 · lr + agenda: +0,031 (⅔ do ganho) · lr de 1e-3 para 1e-2 com queda a zero, ainda no Adam: 0,8646 → 0,8957.

2 · weight decay: +0,004 · Adam → AdamW: 0,8957 → 0,8994. Controla a norma dos pesos, mas o F1 quase não muda.

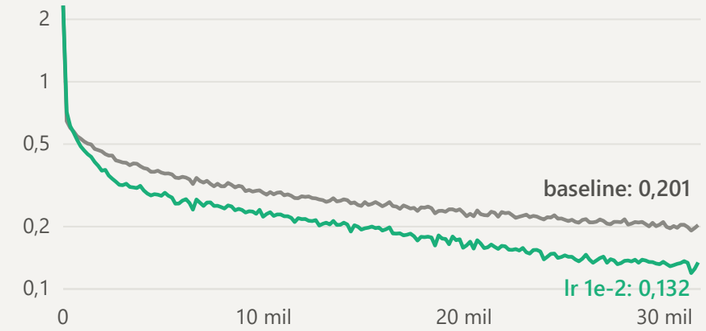
3 · Muon: +0,013 (¼ do ganho) · Muon nas 2 matrizes ocultas: 0,8994 → 0,9121. Só ¼ do ganho, e exige a agenda que decai.

Diagnóstico: lr maior + queda a zero

ρ = ||Δθ||/||θ|| (tamanho do passo, escala log)



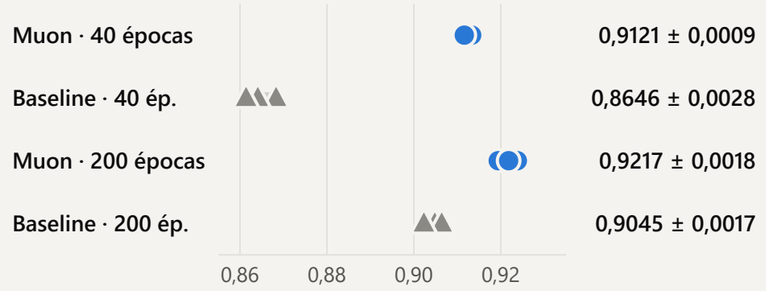
perda de treino (escala log)



- **Baseline:** ρ cai de 1e-3 a **5e-4** e a perda de treino termina em **0,201**: passo pequeno demais, a rede fica **subajustada**.
- **Adam lr 1e-2 + agenda:** ρ sobe a **6e-3** (6× a referência) e a queda do lr o leva a ~0; a perda termina em **0,132**.
- **Duas forças:** lr grande no começo para andar mais; lr perto de 0 no fim para o modelo "assentar" no vale em vez de oscilar com o ruído do mini-lote.

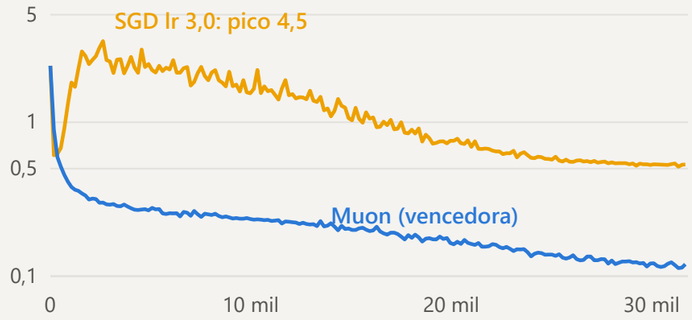
Limites do resultado

Com mais treino o ganho encolhe (1 ponto = 1 semente)



40 épocas: ganho de **+0,0475** (16× o desvio). 200 épocas (3 sementes): **+0,0172** (9× o desvio). A rede ainda não saturou em 40 épocas, então parte do ganho do Muon é **velocidade**. Mesmo assim o ganho continua maior que a variação entre sementes.

Execução instável: SGD com lr 3,0 (perda de treino)



A perda **sobe a 4,5** no passo 1.700 e ||θ|| chega a 271 (817 no fim): **passo grande demais**, e o remédio seria um lr menor. A receita vencedora desce sem picos.